

35. Konceptuální modelování a návrh relační databáze

35. Konceptuální modelování a návrh relační databáze

SZZ okruh 35 (FIT BUT). Od reality přes ER model k normalizovanému schématu DB.

Shrnutí

Konceptuální modelování (ER)

- První krok návrhu DB (etapa analýzy); modeluje **data**, nezávisle na implementaci. Pak **logický** návrh (tabulky) a **fyzický** návrh (uložení).
- **ER model**: **entita** (objekt), **entitní množina** (typ), **atribut** (jednoduchý/složený, jedno/vícehodnotový, NULL, odvozený), **vztah** (kardinalita), **primární klíč** (z kandidátních, minimální).
- **Slabá entita** — závislá na dominantní (PK silné entity + diskriminátor); **generalizace/specializace** (OO princip).

Transformace na tabulky

- **1:1** — zvážit spojení do jedné tabulky, jinak cizí klíč.
- **1:N** — cizí klíč na straně N + případné atributy vazby.
- **M:N** — **vazební tabulka** (složený PK z obou PK + atributy vazby).
- Slabá entita → tabulka se složeným PK (cizí klíč + diskriminátor).

Normalizace

- Cíl: odstranit redundanci, zajistit konzistenci; pro OLTP min. **3. NF**.
- **1. NF** — atomické hodnoty. **2. NF** — 1NF + neklíčové atributy plně závislé na celém klíči. **3. NF** — 2NF + žádná tranzitivní závislost. **BCNF** — přísnější 3NF (kandidátní klíče disjunktní).
- **Funkční závislost** $X \twoheadrightarrow Y$: stejné X → stejné Y.

Relační model a SQL viz [[okruhy/36-relacni-data-sql-transakce]]; diagram tříd jako alternativa ER viz [[okruhy/34-uml]]; relace viz [[okruhy/16-mnoziny-relace-zobrazeni]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

ER model → [[#Konceptuální modelování (ER)]] - *Entita / atribut / vztah / kardinalita?* → Entita = objekt; atribut = vlastnost; vztah = spojení entit; kardinalita = max. počet vztahů jedné entity (1, M). - *Slabá vs. silná entita?* → Slabá nemůže existovat bez dominantní (PK silné + diskriminátor). - *Primární klíč?* → Minimální kandidátní klíč jednoznačně identifikující entitu.

Transformace ER → **tabulky** → [[#Transformace na tabulky]] - *1:N a M:N?* → 1:N cizí klíč na straně N; M:N vazební tabulka se složeným klíčem.

Normalizace □ [[#Normalizace]] - 1./2./3. NF? □ 1NF atomické hodnoty; 2NF plná závislost na klíči; 3NF bez tranzitivní závislosti. - *Funkční závislost?* □ $X \rightarrow Y$: dvě n-tice se stejným X mají stejné Y. - *Proč normalizovat?* □ Odstranění redundance a anomálií, lepší konzistence.

Plné znění (ke studiu)

Konceptuální modelování

Konceptuální modelování je první krok při **návahu uložení dat v databázi** (návahu databáze). Patří do etapy **analýzy požadavků**. Jeho cílem je analyzovat požadavky **na data**, která budou **uložena v databázi**. Zabývá se **modelováním reality** (objektů a jejich vlastností, které potřebujeme ukládat). Snaží se **nebýt** ovlivněno budoucími prostředky řešení. Umožňuje **sjednotit chápání** aplikace mezi uživateli (zákazníky), analytiky a programátory. Výsledek je **použit při logickém návrhu databáze** (jednotlivých tabulek) a slouží také jako **dokumentace**. Obvykle se provádí graficky pomocí:

- **ER modelů** popsaných ER diagramy: jedná se o **strukturovaný** přístup. Při strukturovaném návrhu lze však také použít OOP při implementaci IS.
- **diagramu tříd**: jedná se o objektově orientovaný přístup.

Logický návrh [[media/szz-35/media/image3.png]]

Cílem je navrhnout **strukturu databáze** (strukturu jednotlivých tabulek). Popisuje, jak jsou **data uložena** v databázi, **vztahy** mezi nimi a **integritní omezení** tak, aby neexistovala redundance a struktura dat v databázi byla co nejjednodušší.

Fyzický návrh Jde o **fyzické uložení dat** (sekvenční soubor, indexy, cluster, ...). Musí se navrhnout vzhledem k **SŘBD** pro efektivní přístup.

Entity-Relationship model (Entity-Relationship diagram)

ER model je založen na chápání světa jako množiny základních objektů - **entit** (Entity) a vztahů (Relations) mezi nimi. Popisuje data **staticky** ("v klidu"). **Neukazuje**, jaké **operace** s daty budou probíhat. Někdy se označuje také

jako ERA – třetím základním prvkem modelu jsou **atributy** (Attributes) jednotlivých entit nebo vztahů.

Entita Objekt reálného světa rozlišitelný od jiných objektů, o níž chceme mít informace v DB. Jedná se o **konkrétní objekt**, např. klient s ID 25.

Entitní množina (typ entity) Definuje typ/**množinu entit**, které **sdílí tytéž vlastnosti** neboli atributy. Jedná se skupiny stejných objektů, např. klient, bankovní účet, ...

Atribut Vlastnost entity nebo vztahu, která nás v kontextu daného problému **zajímá a jejíž hodnotu chceme mít v DB** uloženu. Atributy mohou být:

- **jednoduché** (PSČ) a **složené** (celá adresa),
- **jednohodnotové** (popis - objekt má obvykle pouze jeden popis) a **vícehodnotové** (telefon - můžeme požadovat uložení více tel. čísel),
- **povolující prázdnou hodnotu** - NULL (hodnota neexistuje nebo může existovat, ale mi jí zatím neznáme, např. popis) a **nepovolující prázdnou hodnotu** (objekt nemůže existovat bez této hodnoty - např. název, ID, ...),

- **odvozené** (věk je odvozený od data narození) v **OLTP** ne, v **OLAP** ano.

Doména atributu Obor hodnot atributu.

Vztah Asociace (**spojení**) mezi dvěma nebo více entitami. Např. klient s číslem klienta K999 vlastní účet s číslem účtu U100.

Primární klíč Jedná se o jeden (můžeme vybrat) z **kandidátních klíčů**, což je atribut nebo množina atributů, který/která je **v dané množině entit unikátní**. Kandidátní klíč musí být **minimální** (nemůže existovat kandidátní klíč, který má méně atributů).

Vztahy

Spojují **entitní množiny** a vyjadřují vztahy mezi nimi.

Stupeň Určuje, kolik je vztahem **propojeno entitních množin**.

Kardinalita [[media/szz-35/media/image15.png]]

Udává **maximální počet vztahů** daného typu, ve kterých může **participovat jedna entita** (1, M, případně přesněji) z entitní množiny. Jedná se o vlastnost **každého “konce”** vztahu. **Pro návrh schématu databáze je rozhodující správné určení maximální kardinality**. Kardinalitu značíme znaky (0, 1, *) nebo pomocí symbolů na obrázku (Crow's foot notation).

[[media/szz-35/media/image8.png]]

Atributy vztahu **Rozvíjí vztah, v databázi** ale musí být uloženy **ve zvláštní vazební tabulce**. Převádíme je na následující tabulku (druhé schéma).

Slabé a silné entity

[[media/szz-35/media/image10.png]]

- **silná entita** může existovat nezávisle na ostatních,
- **slabá entita** je závislá na jiném **jednom dominantním** typu entity. Slabé entity jsou identifikovány **primárním klíčem silné entity** a **diskriminátorem** - dílčím klíčem. Primární klíč silné entity slouží jako **cizí klíč**. Nemohou existovat samy o sobě a při zániku dominantní entity také zanikají.

Generalizace a specializace

[[media/szz-35/media/image4.png]]

Vychází z principů OOP. Umožňuje rozšiřovat entity o **další atributy** (specializované entity) a současně **sdílet jiné** (obecná entita). Existovat sama o sobě může i obecná entita. Primární klíč odvozené (specializované) entity je **stejný** jako ten obecné.

Jména/Názvy

[[media/szz-35/media/image9.png]]

- **srozumitelná**, musí vyjadřovat význam typů entit a vztahů,
- **typ entit**: podstatná jména,

- **typ vztahů:** slovesa, předložky,
- je-li jméno typu vztahu jasné ze jmen typů entit, není nutné ho uvádět,
- při několika **různých typech vztahů** mezi **stejnými** entitními množinami je **nutné** použít jméno vztahu.

Rozdíl ER diagramu a diagramu tříd

[[media/szz-35/media/image14.png]]

Návrh relační databáze

Schéma relační databáze lze získat jedním z následujících dvou způsobů:

- vytvořením **konceptuálního modelu** a jeho transformací (transformací ER diagramu),
- **použitím normalizace** s tím, že na počátku předpokládáme, že **všechny informace** budou uloženy **v jedné tabulce** a tu normalizujeme.

Transformace ER diagramu na tabulky relační databáze

Jedná se o další krok při návrhu databáze. Jde o převod z **konceptuálního návrhu** na **návrh logický**. Tento způsob se používá častěji než normalizace, ale současně dbáme na to, aby data byla v požadované normální formě.

- **Vztahy s kardinalitou 1:1:** Ujistíme se, že je opravdu vhodné vytvářet tento vztah a není lepší tabulky **spojit do jedné** (neplatí při vazbě 0..1). Jinak rozšíříme jednu z tabulek o **cizí klíč** do druhé a přidáme do ní **atributy vztahu**.
- **Vztahy s kardinalitou 1:N:** Záznamy tabulky (o vztahu s kardinalitou N), které jsou ve vztahu **pouze s jedním záznamem** (o vztahu s kardinalitou 1) musí obsahovat **cizí klíč** do druhé tabulky. Tyto záznamy mohou být také **rozšířeny o atributy vazby**, protože na libovolný záznam druhé tabulky může takto **odkazovat M záznamů první tabulky** (kde každý bude mít své vlastní atributy vztahu).
- **Vztahy s kardinalitou M:N:** Reprezentujeme přidáním do databáze (vazební) tabulky. Primární klíč se obvykle volí jako **složený klíč z primárních klíčů původních tabulek**. Do vazební tabulky se také přidávají atributy vazby. [[media/szz-35/media/image7.png]]
- **Vztah vyššího stupně:** Z naprosté většiny vyžaduje tvorbu vazební tabulky, která funguje na principu jako vazební tabulka při binární vazbě M:N.
- **Vztah slabé entitní množiny:** Tato vazba je **vždy 1:M** a provádí se stejně s tím, že tabulka reprezentující slabou entitní množinu má složený primární klíč z cizího klíče do **dominantní** tabulky a **diskriminátoru**.
- **Generalizace a specializace:** Lze transformací do tabulek databáze řešit čtyřmi způsoby
 1. Vytvoření obecné tabulky se sdílenými atributy a tabulek specializovaných, které obsahují pouze specifické atributy. Mají **stejný primární klíč**, který je **současně cizím klíčem** do obecné tabulky.
 - **výhody:** Tato varianta je vhodná pro **disjunktní i překrývající se** specializace a pro **specializaci úplnou i částečnou**. Vhodné také při očekávání **nových specializací**.
 - **nevýhody:** nutnost provádět operaci **spojování**.

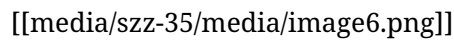
2. Vytvoření dvou nezávislých tabulek, které budou obsahovat atributy obecné entitní množiny a poté každá atributy své specializované entitní množiny. Tento způsob **neumožňuje vytváření pouze obecných entit** (musely by se ukládat do jedné ze specializovaných tabulek, což není vhodné).
 - **výhody:** Není potřeba provádět operace spojování - join.
 - **nevýhody:** specializace musí být **úplná** (nemůže existovat obecný záznam) a **disjunktní, nelze** provádět **společné** operace nad obecnými daty.
3. Vytvoří se pouze **jedna tabulka**, která obsahuje **obecné atributy i atributy všech specializací**. Jednotlivé specializace pak můžeme rozlišovat na základě **testu prázdné hodnoty** ve specializovaných sloupcích (pokud je to možné a některý ze sloupců specializace nemůže být prázdný) nebo přidáním speciálního sloupce - **diskriminátoru**.
 - **výhody:** není potřeba provádět spojování,
 - **nevýhody:** může vést na velké tabulky a mnoho prázdných hodnot, zejména při větším počtu disjunktních specializací. V tomto případě je také vhodné přidat další sloupec identifikující jednotlivé specializace, aby se nemusel provádět složitý test na prázdné hodnoty.
4. Vytvoření tabulky pro **obecnou entitní množinu** a **jedné tabulky pro všechny specializace**, v této tabulce lze opět zavést **diskriminátor**. Primární klíč v obou tabulkách bude stejný a slouží i jako cizí klíč.
 - **výhody:** Lze rychle pracovat s daty obecné entitní množiny.
 - **nevýhody:** spojování, prázdné hodnoty, test na prázdnost sloupců.

Normalizace

Normalizace je druhý způsob, jak můžeme postupovat při návrhu relační databáze pro uložení požadovaných dat. Tento způsob (pokud provedený správně) **zajišťuje, že databáze nebude trpět nedostatky špatného návrhu**. Při použití normalizace se vychází z **jedné tabulky**, ve které jsou uložena všechna data. Pokud není tabulka navržena správně, je **nutné ji rozdělit** na dvě nebo více tabulek jednodušších. To, jestli je tabulka navržena správně nebo není určuje normální forma, podle které tabulku konstruujeme. Pro **OLTP** systémy požadujeme alespoň **3. NF**.

1. Normální forma

Schéma relace je v 1. normální formě, pokud její atributy obsahují pouze **atomické** (skalární) hodnoty, které nelze dále dělit. Typickým příkladem převodu tabulky do 1. normální formy je **převod atributu adresa** na několik atributů jako: **země, město, PSČ, ulice, číslo popisné, ...** Na obrázku u příkladu s telefony je také možným řešením **vytvořit vazbu 1:M**.  



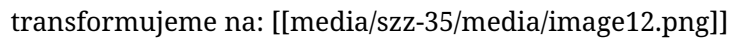
2. Normální forma

Schéma relace (tabulka) je v druhé normální formě, pokud je v **1. NF** a **každý její neklíčový atribut je plně funkčně závislý na každém kandidátním klíči relace** (musí být závislý na všech atributech složeného klíče). Převod do 2. NF zajišťujeme rozdělením tabulky na více tabulek a propojením pomocí cizích klíčů. Na obrázku tímto **cizím klíčem** je **Č_ÚČTU**. 

3. Normální forma



Schéma relace je ve třetí normální formě, právě když je ve 2NF a **neexistuje žádný neklíčový atribut, který je tranzitivně závislý na některém kandidátním klíči relace**. Tranzitivní závislost je závislost mezi minimálně dvěma atributy a klíčem, kde **jeden atribut je funkčně závislý na klíči a druhý atribut je funkčně závislý na prvním atributu**. Řešíme tak, že vytvoříme novou tabulku s těmito atributy, kde první atribut je klíčem a v původní tabulce zůstane pouze první atribut jako cizí klíč do nové tabulky. 

transformujeme na: 



Boyce-Coddova normální forma

Schéma relace je v Boyce-Coddově normální formě, pokud je ve 3. normální formě a v relaci existuje **pouze jeden** kandidátní klíč, nebo jich existuje více, ale jsou **disjunktní** (nemají společný atribut). Jinak řečeno, pokud v relaci **existují dva** (nebo více) **složené** kandidátní klíče, které **sdílí nějaký atribut**, není relace v BCNF. Opět řešíme tak, že relaci rozdělíme na dvě (2 tabulky) a některý atribut použijeme jako cizí klíč.

Funkční závislost



Pokud je **Y** funkčně závislé na **X** ($X \rightarrow Y$), pak se **nemůže stát** aby **2 řádky** mající stejnou hodnotu **X** měly různou hodnotu **Y**.

- Příklad: **Datum narození** je funkčně závislé na **rodném čísle** - nemůže se stát že u 2 záznamů **se stejným rodným číslem** bude rozdílné datum narození. Uvažujeme, že rodné číslo není primární klíč.

Zdroje

- SZZ okruh 35 — studijní materiály FIT BUT (szz-35.docx). Obrázky: media/szz-35/.

Navigace: [\[\[index|• Přehled okruhů\]\]](#) · [\[\[okruhy/34-uml|34. Jazyk UML\]\]](#) · další: [\[\[okruhy/36-relacni-data-sql-transakce|36. Strukturovaná data, SQL, transakce\]\]](#) 